

RECURSIVIDADE

Do caso-base à Busca Binária

Recursividade não é apenas repetição: é uma forma de decompor problemas.

1. Objetivos de aprendizagem

Ao final deste estudo, você deverá ser capaz de:

- explicar o conceito de recursividade;
- identificar caso-base e chamada recursiva;
- explicar por que o problema precisa diminuir a cada chamada;
- acompanhar a pilha de chamadas (call stack);
- distinguir as fases de chamada e retorno;
- diferenciar recursividade de iteração;
- entender que recursividade não significa automaticamente maior eficiência;
- reconhecer problemas naturalmente recursivos;
- relacionar recursividade a Dividir e Conquistar e à possibilidade de paralelismo;
- implementar e analisar uma Busca Binária recursiva.

2. Por que aprender recursividade?

Se já existem `for` e `while`, pode parecer desnecessário aprender outra forma de repetir operações. Essa interpretação é limitada. O principal valor da recursividade não é substituir laços, mas permitir que um problema seja descrito em termos de versões menores do próprio problema.

```
PROBLEMA
  ↓
problema menor
  ↓
problema menor
  ↓
CASO-BASE
```

Essa forma de raciocínio aparece em árvores, sistemas de arquivos, busca, backtracking e em algoritmos baseados em Dividir e Conquistar, como Merge Sort e Quick Sort.

Pergunta central: consigo transformar este problema em uma versão menor do mesmo problema?

3. Onde esse conhecimento será usado?

- Árvores e outras estruturas hierárquicas.
- Busca Binária.
- Merge Sort e Quick Sort.
- Algoritmos de Dividir e Conquistar.
- Backtracking, labirintos, combinações e problemas de busca.
- Decomposição de tarefas que, quando independentes, pode permitir paralelismo.

Importante: recursividade não é sinônimo de paralelismo nem de maior velocidade. Ela é um mecanismo de expressão e decomposição. A eficiência depende da estratégia algorítmica adotada.

4. Primeiro exemplo: contagem regressiva

Uma solução iterativa conhecida:

```
def contagem(n):  
    while n > 0:  
        print(n)  
        n -= 1  
  
contagem(5)
```

Podemos tentar resolver o mesmo problema fazendo a função chamar a si própria:

```
def contagem(n):  
    print(n)  
    contagem(n - 1)
```

O problema é que essa função não possui uma condição de parada. Ela continuará chamando `contagem(0)`, `contagem(-1)`, `contagem(-2)` e assim sucessivamente.

5. Caso-base e passo recursivo

```
def contagem(n):  
    if n == 0:  
        return  
  
    print(n)  
    contagem(n - 1)
```

Agora existem dois componentes essenciais:

5.1 Caso-base

```
if n == 0:  
    return
```

Define quando não é necessária uma nova chamada.

5.2 Passo recursivo

```
contagem(n - 1)
```

Transforma o problema atual em uma versão menor do mesmo problema.

Uma função recursiva precisa caminhar em direção ao caso-base.

6. Definição de recursividade

Recursividade é uma técnica na qual uma função resolve um problema chamando a si própria para resolver uma versão menor desse mesmo problema.

```
def resolver(problema):  
    if caso_base:  
        return resultado  
  
    problema_menor = reduzir(problema)  
    return resolver(problema_menor)
```

Ao analisar qualquer função recursiva, pergunte:

- Qual é o caso-base?
- Qual é a chamada recursiva?
- Como o problema fica menor?
- O que acontece quando a chamada recursiva retorna?

7. Ter caso-base não é suficiente

```
def contagem(n):  
    if n == 0:  
        return  
  
    print(n)  
    contagem(n)
```

Embora exista um caso-base, n nunca muda. Se a função começar com $n = 5$, continuará chamando `contagem(5)`. Logo, nunca alcançará $n == 0$.

```
contagem(5)  
↓  
contagem(5)  
↓  
contagem(5)  
↓  
...
```

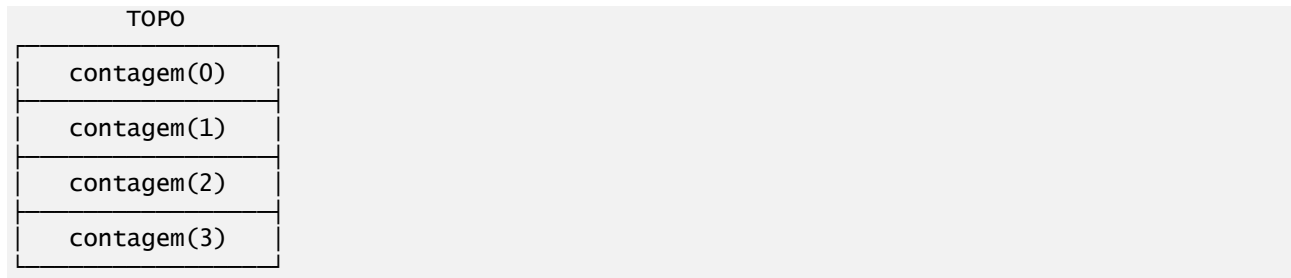
8. O que acontece com as chamadas anteriores?

Quando `contagem(5)` chama `contagem(4)`, a primeira chamada ainda não terminou. Ela fica aguardando. Cada chamada possui seus próprios parâmetros, variáveis locais e ponto de retorno.

```
contagem(5)  
└─ contagem(4)  
    └─ contagem(3)  
        └─ contagem(2)  
            └─ contagem(1)  
                └─ contagem(0)
```

9. A pilha de chamadas (Call Stack)

As chamadas pendentes são mantidas na pilha de execução. Pilhas seguem a regra LIFO: Last In, First Out (o último a entrar é o primeiro a sair).



Quando `contagem(0)` executa `return`, ela sai da pilha. Depois termina `contagem(1)`, depois `contagem(2)`, e assim por diante.

10. Descida e retorno da recursão

```
def exemplo(n):
    if n == 0:
        return
    print("Entrando:", n)
    exemplo(n - 1)
    print("Saindo:", n)
exemplo(3)
```

Saída:

```
Entrando: 3
Entrando: 2
Entrando: 1
Saindo: 1
Saindo: 2
Saindo: 3
```

A recursão possui uma fase de descida, até o caso-base, e uma fase de retorno, quando as chamadas pendentes continuam sua execução.

```
DESCIDA:
3 → 2 → 1 → 0

CASO-BASE

RETORNO:
0 → 1 → 2 → 3
```

11. Recursividade com retorno de valores

Considere a soma de 1 até `n`. Podemos escrever:

```
def soma(n):
    if n == 0:
        return 0

    return n + soma(n - 1)
```

Para `soma(4)`:

```
soma(4) = 4 + soma(3)
        = 4 + 3 + soma(2)
        = 4 + 3 + 2 + soma(1)
        = 4 + 3 + 2 + 1 + soma(0)
```

```
soma(0) = 0
```

Retorno:

```
soma(1) = 1
soma(2) = 3
soma(3) = 6
soma(4) = 10
```

12. Exemplo clássico: fatorial

Matematicamente:

$$n! = n \times (n - 1)! \quad \text{e} \quad 0! = 1$$

```
def fatorial(n):
    if n == 0:
        return 1
    return n * fatorial(n - 1)
```

Para fatorial(4), as chamadas descem até fatorial(0) e depois os resultados retornam:

```
fatorial(0) = 1
fatorial(1) = 1
fatorial(2) = 2
fatorial(3) = 6
fatorial(4) = 24
```

13. A mudança na forma de pensar

Em uma solução iterativa, é comum pensar: “qual é o próximo passo?”. Em uma solução recursiva, a pergunta mais útil é: “se eu soubesse resolver uma versão menor desse problema, como usaria essa solução para resolver o problema atual?”

Recursão é, sobretudo, uma forma de raciocinar sobre decomposição.

14. Recursão substitui loops?

Não. Compare:

```
def soma_iterativa(n):
    soma = 0
    for i in range(n + 1):
        soma += i
    return soma
def soma_recursiva(n):
    if n == 0:
        return 0
    return n + soma_recursiva(n - 1)
```

Para esse problema, a solução iterativa é simples e evita várias chamadas de função. Recursividade pode ter overhead e consumir espaço na pilha. Em Python, uma recursão excessivamente profunda pode gerar `RecursionError`.

RECURSIVIDADE ≠ MAIOR EFICIÊNCIA

15. Problemas naturalmente recursivos

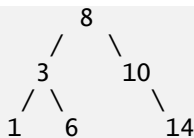
A recursividade torna-se especialmente útil quando a própria estrutura do problema é hierárquica ou contém subestruturas do mesmo tipo.

15.1 Diretórios

```
Documentos
├── Faculdade
│   ├── DSA
│   │   ├── aula1.pdf
│   │   └── aula2.pdf
│   └── Python
│       └── exercicios.py
└── Pessoal
    └── contas.pdf

def percorrer(diretorio):
    for item in diretorio:
        if item. eh_arquivo():
            print(item)
        else:
            percorrer(item)
```

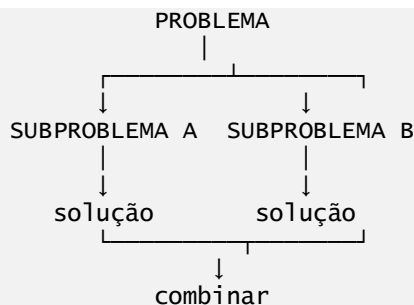
15.2 Árvores



Cada subárvore é novamente uma árvore. Por isso, percursos e buscas em árvores frequentemente possuem soluções recursivas naturais.

16. Decomposição em vários subproblemas

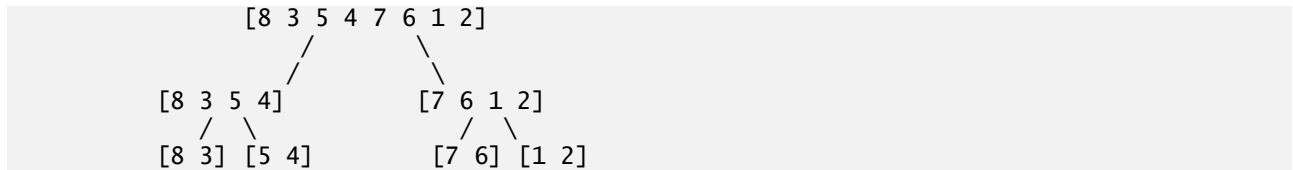
Até agora vimos uma cadeia de chamadas. Entretanto, alguns problemas podem ser divididos em mais de um subproblema.



Quando os subproblemas são do mesmo tipo do problema original, podemos resolvê-los recursivamente. Essa é uma das bases do paradigma Dividir e Conquistar.

17. Prévia: Merge Sort

Sem implementar o algoritmo ainda, observe sua estrutura de divisão:



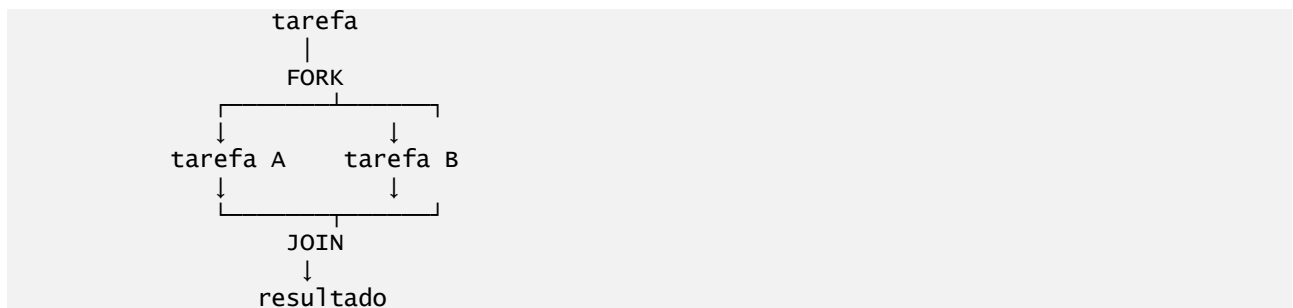
As duas metades são problemas de ordenação. Posteriormente veremos que Merge Sort resolve cada metade recursivamente e combina os resultados.

18. Recursividade e paralelismo

A recursividade não cria paralelismo automaticamente. Porém, uma decomposição recursiva pode revelar subproblemas independentes. Se houver recursos computacionais adequados, tarefas independentes podem, em certos casos, ser executadas simultaneamente.



Esse padrão está relacionado ao modelo fork/join: uma tarefa é dividida (fork), as partes são processadas e depois sincronizadas (join).



Recursividade → decomposição → subproblemas independentes → possibilidade de paralelismo

19. Nem toda recursão é paralelizável

Uma cadeia como contagem($n - 1$) possui dependência sequencial e não cria duas tarefas necessárias e independentes.

```
5
↓
4
↓
3
↓
2
↓
1
```

A possibilidade de paralelismo depende da estrutura de dependências do problema, e não simplesmente da presença de chamadas recursivas.

20. Da recursividade à Busca Binária

A Busca Binária é um excelente exemplo porque já possui uma estrutura de redução do problema: a cada etapa, metade do espaço de busca é descartada.

```
n
↓
n/2
↓
n/4
↓
n/8
↓
...
```

Considere o vetor ordenado:

```
[3, 8, 12, 17, 25, 31, 42, 56, 68, 74, 89]
```

A cada comparação com o elemento central, escolhemos apenas a metade que ainda pode conter o valor procurado.

21. Busca Binária iterativa

```
def busca_binaria(lista, valor):
    inicio = 0
    fim = len(lista) - 1
    while inicio <= fim:
        meio = (inicio + fim) // 2

        if lista[meio] == valor:
            return meio
        if valor < lista[meio]:
            fim = meio - 1
        else:
            inicio = meio + 1
    return -1
```

O while repete sempre o mesmo problema: buscar um valor entre os índices início e fim. A cada repetição, esse intervalo fica menor.

22. Construindo a Busca Binária recursiva

Aplicamos as perguntas fundamentais.

22.1 Caso-base

```
if inicio > fim:
    return -1
```

Não existe mais região de busca.

22.2 Elemento encontrado

```
meio = (inicio + fim) // 2

if lista[meio] == valor:
    return meio
```

22.3 Redução do problema

```
if valor < lista[meio]:
    return busca_binaria(lista, valor, inicio, meio - 1)

return busca_binaria(lista, valor, meio + 1, fim)
```

23. Busca Binária recursiva completa

```
def busca_binaria(lista, valor, inicio, fim):
    # Caso-base: não há mais elementos
    if inicio > fim:
        return -1

    meio = (inicio + fim) // 2

    # Encontrou o elemento
    if lista[meio] == valor:
        return meio

    # Busca na metade esquerda
    if valor < lista[meio]:
        return busca_binaria(
            lista,
            valor,
            inicio,
            meio - 1
        )

    # Busca na metade direita
    return busca_binaria(
        lista,
        valor,
        meio + 1,
        fim
    )

lista = [3, 8, 12, 17, 25, 31, 42, 56, 68, 74, 89]

posicao = busca_binaria(
    lista,
    56,
    0,
    len(lista) - 1
)
```

```
print(posicao)
```

24. Rastreando a execução

Considere:

```
[2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 72, 81, 91, 95, 99]
```

Buscando 81:

```
1ª chamada:
inicio = 0
fim    = 14
meio   = 7
lista[7] = 45

81 > 45
→ busca_binaria(..., 8, 14)

2ª chamada:
inicio = 8
fim    = 14
meio   = 11
lista[11] = 81
→ encontrou na posição 11
```

25. Complexidade da Busca Binária

A versão recursiva não é assintoticamente mais rápida que a iterativa. Ambas eliminam metade do espaço de busca a cada passo.

$$T(n) = T(n/2) + O(1) \Rightarrow O(\log n)$$

Característica	Iterativa	Recursiva
Tempo	$O(\log n)$	$O(\log n)$
Espaço auxiliar	$O(1)$	$O(\log n)$
Usa call stack para repetir a busca	Não	Sim
Exige vetor ordenado	Sim	Sim

A eficiência $O(\log n)$ vem da redução pela metade, não da recursividade.

26. A Busca Binária pode ser paralelizada dessa forma?

Na Busca Binária, depois de comparar com o elemento central, escolhemos esquerda OU direita. Não precisamos resolver simultaneamente as duas metades.

```
Busca Binária:
problema
  ↓
uma metade
  ↓
uma metade
```

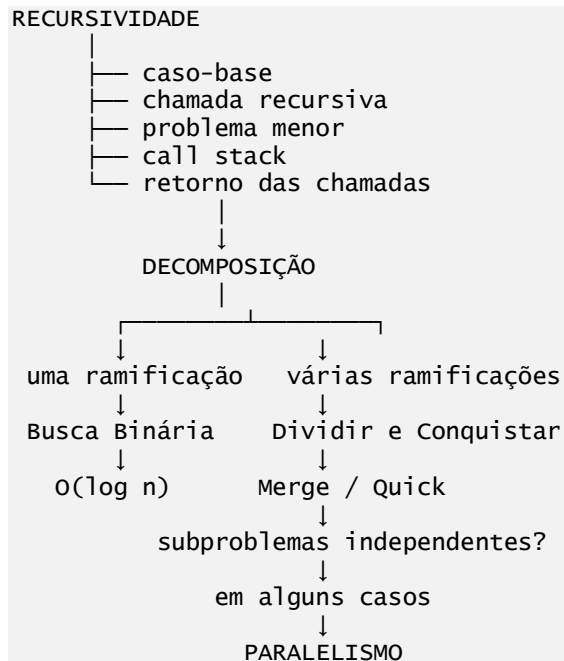
```

↓
...
Merge Sort (conceitualmente):
problema
├── metade esquerda
└── metade direita

```

Por isso, o simples fato de um algoritmo ser recursivo não significa que ele ofereça paralelismo útil. É necessário analisar a independência entre os subproblemas.

27. Síntese da aula



28. Checklist para analisar uma função recursiva

1. Qual é o caso-base?
2. Qual é a chamada recursiva?
3. Como o problema fica menor?
4. O que acontece quando a chamada retorna?
5. Existem diferentes subproblemas independentes?

29. Exercícios de fixação

1. Identifique o caso-base e a chamada recursiva da função contagem apresentada na apostila.
2. Explique por que uma função que chama a si própria com o mesmo argumento pode nunca terminar.
3. Desenhe a pilha de chamadas para exemplo(4).

4. Mostre a descida e o retorno de soma(5).
5. Calcule manualmente fatorial(5) usando a expansão recursiva.
6. Explique por que recursividade não significa automaticamente maior eficiência.
7. Dê um exemplo de estrutura de dados cuja própria organização seja naturalmente recursiva.
8. Explique em que condição uma decomposição recursiva pode abrir possibilidade de paralelismo.
9. Reescreva a Busca Binária recursiva e identifique o caso-base.
10. Explique por que as versões iterativa e recursiva da Busca Binária continuam sendo $O(\log n)$.
11. Explique por que a Busca Binária não possui, em sua forma usual, duas metades úteis para processar simultaneamente.

30. Para a próxima aula

A Busca Binária reduz o problema e escolhe apenas uma das metades. O próximo passo é estudar problemas nos quais precisamos resolver mais de uma parte. Isso conduz ao paradigma Dividir e Conquistar e, em seguida, ao Merge Sort e ao Quick Sort.

Próxima questão: e se precisarmos resolver recursivamente as duas metades?